

=====

```

+-----+
| type.__annotations__          | A dictionary containing *variable
annotations* |
|                               | collected during class body execution.
See also: |
|                               | "__annotations__ attributes". For best
practices |
|                               | on working with "__annotations__",
please see |
|                               | "annotationlib". Use
|                               |
|                               | "annotationlib.get_annotations()"
instead of |
|                               | accessing this attribute directly.
Warning: |
|                               | Accessing the "__annotations__"
attribute directly |
|                               | on a class object may return annotations
for the |
|                               | wrong class, specifically in certain
cases where |
|                               | the class, its base class, or a
metaclass is |
|                               | defined under "from __future__ import
|                               | annotations". See **749** for
details.This |
|                               | attribute does not exist on certain
builtin |
|                               | classes. On user-defined classes without
|                               |
|                               | "__annotations__", it is an empty
dictionary. |
|                               | Changed in version 3.14: Annotations are
now |
|                               | lazily evaluated. See **PEP 649**.
```

```

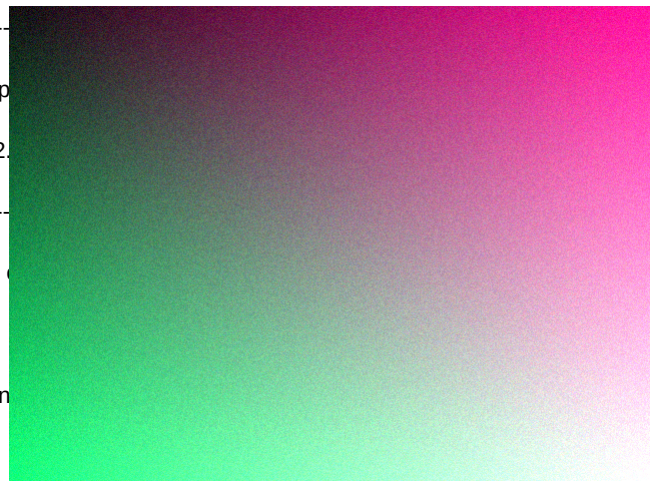
+-----+
| type.__annotate__()          | The *annotate function* for this class,
or "None" |
|                               | if the class has no annotations. See
also: |
|                               | "__annotate__ attributes". Added in
version 3.14. |
```

```

+-----+
| type.__type_params__         | A "tuple" containing the type
of a |
|                               | generic class. Added in version 3.12.
|
```

```

+-----+
| type.__static_attributes__   | A "tuple" containing names of
of this |
|                               | class which are assigned through
"self.X" from any |
|                               | function in its body. Added in version
3.13. |
```



```

+-----+-----+
-----+
| type.__firstlineno__          | The line number of the first line of the
class |                          | definition, including decorators.
|                                     |
Setting the |                                     |
|                                     | "__module__" attribute removes the
|                                     |
|                                     | "__firstlineno__" item from the type-s
dictionary. |
|                                     | Added in version 3.13.
|
|
+-----+-----+
-----+
| type.__mro__                  | The "tuple" of classes that are
considered when |
|                               | looking for base classes during method
resolution. |
+-----+-----+
-----+

```

Special methods

In addition to the special attributes described above, all Python classes also have the following two methods available:

`type.mro()`

This method can be overridden by a metaclass to customize the method resolution order for its instances. It is called at class instantiation, and its result is stored in "`__mro__`".

`type.__subclasses__()`

Each class keeps a list of weak references to its immediate subclasses. This method returns a list of all those references still alive. The list is in definition order. Example:

```

>>> class A: pass
>>> class B(A): pass
>>> A.__subclasses__()
[<class 'B'>]

```

Class instances

=====

A class instance is created by calling a class object (see above). A class instance has a namespace implemented as a dictionary which is the first place in which attribute references are searched. When an attribute is not found there, and the instance's class has an attribute by that name, the search continues with the class attributes. If a class attribute is found that is a user-defined function object, it is transformed into an instance method object whose "`__self__`" attribute is the instance. Static method and class method objects are also transformed; see above under "Classes". See section Implementing Descriptors for another way in which attributes of a class retrieved via its instances may differ from the objects actually stored in the class's "`__dict__`". If no class attribute is found, and the object's class has a "`__getattr__()`" method, that is

called to satisfy the lookup.

Attribute assignments and deletions update the instance's dictionary, never a class's dictionary. If the class has a "`__setattr__()`" or "`__delattr__()`" method, this is called instead of updating the instance dictionary directly.

Class instances can pretend to be numbers, sequences, or mappings if they have methods with certain special names. See section [Special method names](#).

Special attributes

`object.__class__`

The class to which a class instance belongs.

`object.__dict__`

A dictionary or other mapping object used to store an object's (writable) attributes. Not all instances have a "`__dict__`" attribute; see the section on `__slots__` for more details.

I/O objects (also known as file objects)

A *file object* represents an open file. Various shortcuts are available to create file objects: the "`open()`" built-in function, and also "`os.popen()`", "`os.fdopen()`", and the "`makefile()`" method of socket objects (and perhaps by other functions or methods provided by extension modules).

File objects implement common methods, listed below, to simplify usage in generic code. They are expected to be [With Statement Context Managers](#).

The objects "`sys.stdin`", "`sys.stdout`" and "`sys.stderr`" are initialized to file objects corresponding to the interpreter's standard input, output and error streams; they are all open in text mode and therefore follow the interface defined by the "`io.TextIOBase`" abstract class.

`file.read(size=-1, /)`

Retrieve up to *size* data from the file. As a convenience if *size* is unspecified or -1 retrieve all data available.

`file.write(data, /)`

Store *data* to the file.

`file.close()`

Flush any buffers and close the underlying file.

Internal types

A few types used internally by the interpreter are exposed to the


```

|
+-----+-----+
-----+
| codeobject.co_varnames          | A "tuple" containing the names of the
local      |
|                               | variables in the function (starting with
the        |
|                               | parameter names)
|
+-----+-----+
-----+
| codeobject.co_cellvars          | A "tuple" containing the names of local
variables |
|                               | that are referenced from at least one
*nested    |
|                               | scope* inside the function
|
+-----+-----+
-----+
| codeobject.co_freevars          | A "tuple" containing the names of *free
(closure) |
|                               | variables* that a *nested scope*
references in an |
|                               | outer scope. See also
"function.__closure__". |
|                               | Note: references to global and builtin
names are |
|                               | *not* included.
|
+-----+-----+
-----+
| codeobject.co_code              | A string representing the sequence of
*bytecode* |
|                               | instructions in the function
|
+-----+-----+
-----+
| codeobject.co_consts            | A "tuple" containing the literals used
by the     |
|                               | *bytecode* in the function
|
+-----+-----+
-----+
| codeobject.co_names             | A "tuple" containing the names used by
the        |
|                               | *bytecode* in the function
|
+-----+-----+
-----+
| codeobject.co_filename          | The name of the file from w
was        |
|                               | compiled
|
+-----+-----+
-----+
| codeobject.co_firstlineno       | The line number of the first l
function |
+-----+-----+
-----+
| codeobject.co_lnotab            | A string encoding the mappin
*bytecode* |
|                               | offsets to line numbers. For details,
|

```

```

see the |
| source code of the interpreter.
Deprecated since |
| version 3.12: This attribute of code
objects is |
| deprecated, and may be removed in Python
3.15. |
+-----+
-----+
| codeobject.co_stacksize | The required stack size of the code
object |
+-----+
-----+
| codeobject.co_flags | An "integer" encoding a number of flags
for the |
| interpreter.
|
+-----+
-----+

```

The following flag bits are defined for "co_flags": bit "0x04" is set if the function uses the "*arguments" syntax to accept an arbitrary number of positional arguments; bit "0x08" is set if the function uses the "***keywords" syntax to accept arbitrary keyword arguments; bit "0x20" is set if the function is a generator. See Code Objects Bit Flags for details on the semantics of each flags that might be present.

Future feature declarations (for example, "from __future__ import division") also use bits in "co_flags" to indicate whether a code object was compiled with a particular feature enabled. See "compiler_flag".

Other bits in "co_flags" are reserved for internal use.

If a code object represents a function and has a docstring, the "CO_HAS_DOCSTRING" bit is set in "co_flags" and the first item in "co_consts" is the docstring of the function.

Methods on code objects

codeobject.co_positions()

Returns an iterable over the source code positions of each *bytecode* instruction in the code object.

The iterator returns "tuple"s containing the "(start_line, end_line, start_column, end_column)". The *i-th* tuple corresponds to the position of the source code that compiled to the *i-th* code unit. Column information is 0-indexed utf-8 byte offsets on the given source line.

This positional information can be missing. A non-exhaustive lists of cases where this may happen:

- * Running the interpreter with "-X" "no_debug_ranges".
- * Loading a pyc file compiled while using "-X" "no_debug_ranges".
- * Position tuples corresponding to artificial instructions.

* Line and column numbers that can't be represented due to implementation specific limitations.

When this occurs, some or all of the tuple elements can be "None".

Added in version 3.11.

Note:

This feature requires storing column positions in code objects which may result in a small increase of disk usage of compiled Python files or interpreter memory usage. To avoid storing the extra information and/or deactivate printing the extra traceback information, the "-X" "no_debug_ranges" command line flag or the "PYTHONNODEBUGRANGES" environment variable can be used.

`codeobject.co_lines()`

Returns an iterator that yields information about successive ranges of **bytecode*s*. Each item yielded is a "(start, end, lineno)" tuple:

* "start" (an "int") represents the offset (inclusive) of the start of the **bytecode** range

* "end" (an "int") represents the offset (exclusive) of the end of the **bytecode** range

* "lineno" is an "int" representing the line number of the **bytecode** range, or "None" if the bytecodes in the given range have no line number

The items yielded will have the following properties:

* The first range yielded will have a "start" of 0.

* The "(start, end)" ranges will be non-decreasing and consecutive. That is, for any pair of "tuple"s, the "start" of the second will be equal to the "end" of the first.

* No range will be backwards: "end >= start" for all triples.

* The last "tuple" yielded will have "end" equal to the size of the **bytecode**.

Zero-width ranges, where "start == end", are allowed. Zero-width ranges are used for lines that are present in the source code, but have been eliminated by the **bytecode** compiler.

Added in version 3.10.

See also:

****PEP 626**** - Precise line numbers for debugging and other tools.
The PEP that introduced the "co_lines()" method.

`codeobject.replace(**kwargs)`

Return a copy of the code object with new values for the specified fields.

Code objects are also supported by the generic function `"copy.replace()"`.

Added in version 3.8.

Frame objects

Frame objects represent execution frames. They may occur in traceback objects, and are also passed to registered trace functions.

Special read-only attributes

```
+-----+-----+
+-----+
| frame.f_back          | Points to the previous stack frame
| (towards the         |
| caller), or "None" if this is the bottom
| stack               |
| frame               |
+-----+-----+
+-----+
| frame.f_code          | The code object being executed in this
| frame               |
| Accessing this attribute raises an
| auditing event      |
| "object.__getattr__" with arguments
| "obj" and           |
| ""f_code"".         |
+-----+-----+
+-----+
| frame.f_locals        | The mapping used by the frame to look up
| local               |
| variables. If the frame refers to an
| *optimized          |
| scope*, this may return a write-through
| proxy               |
| object. Changed in version 3.13: Return
| a proxy             |
| for optimized scopes.
+-----+-----+
+-----+
| frame.f_globals       | The dictionary used by the frame to look
| up global           |
| variables            |
+-----+-----+
+-----+
| frame.f_builtins      | The dictionary used by the frame to look
| up built-in         |
| in (intrinsic) names
+-----+-----+
+-----+
| frame.f_lasti         | The precise instruction of the frame
| object               |
```

```

|                                     | (this is an index into the *bytecode*
string of |
|                                     | the code object)
|
+-----+-----+
-----+
| frame.f_generator                   | The *generator* or *coroutine* object
that owns |
|                                     | this frame, or "None" if the frame is a
normal |
|                                     | function. Added in version 3.14.
|
+-----+-----+
-----+

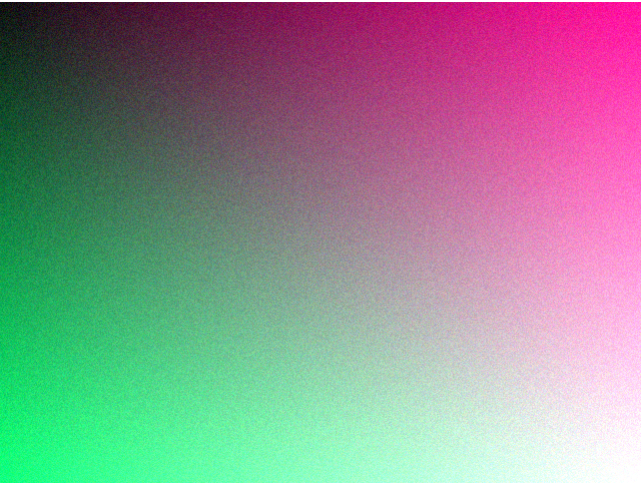
```

Special writable attributes

```

+-----+-----+
-----+
| frame.f_trace                       | If not "None", this is a function called
for |
|                                     | various events during code execution
(this is used |
|                                     | by debuggers). Normally an event is
triggered for |
|                                     | each new source line (see
"f_trace_lines"). |
+-----+-----+
-----+
| frame.f_trace_lines                 | Set this attribute to "False" to disable
|
|                                     | triggering a tracing event for each
source line. |
+-----+-----+
-----+
| frame.f_trace_opcodes               | Set this attribute to "True" to allow
per-opcode |
|                                     | events to be requested. Note that this
may lead to |
|                                     | undefined interpreter behaviour if
exceptions |
|                                     | raised by the trace function escape to
the |
|                                     | function being traced.
|
+-----+-----+
-----+
| frame.f_lineno                       | The current line number of the frame
writing to |
|                                     | this from within a trace function jumps
to the |
|                                     | given line (only for the bottom-most
frame). A |
|                                     | debugger can implement a Jump command
(aka Set |
|                                     | Next Statement) by writing to this
attribute. |
+-----+-----+
-----+

```



Frame object methods

Frame objects support one method:

`frame.clear()`

This method clears all references to local variables held by the frame. Also, if the frame belonged to a *generator*, the generator is finalized. This helps break reference cycles involving frame objects (for example when catching an exception and storing its traceback for later use).

"RuntimeError" is raised if the frame is currently executing or suspended.

Added in version 3.4.

Changed in version 3.13: Attempting to clear a suspended frame raises "RuntimeError" (as has always been the case for executing frames).

Traceback objects

Traceback objects represent the stack trace of an exception. A traceback object is implicitly created when an exception occurs, and may also be explicitly created by calling `types.TracebackType`.

Changed in version 3.7: Traceback objects can now be explicitly instantiated from Python code.

For implicitly created tracebacks, when the search for an exception handler unwinds the execution stack, at each unwound level a traceback object is inserted in front of the current traceback. When an exception handler is entered, the stack trace is made available to the program. (See section The try statement.) It is accessible as the third item of the tuple returned by `sys.exc_info()`, and as the `"__traceback__"` attribute of the caught exception.

When the program contains no suitable handler, the stack trace is written (nicely formatted) to the standard error stream; if the interpreter is interactive, it is also made available to the user as `"sys.last_traceback"`.

For explicitly created tracebacks, it is up to the creator of the traceback to determine how the `"tb_next"` attributes should be linked to form a full stack trace.

Special read-only attributes:

```
+-----+-----+
+-----+
| traceback.tb_frame          | Points to the execution frame of the
current |
|                             | level. Accessing this attribute raises
an      |
|                             | auditing event "object.__getattr__" with
arguments |
|                             | "obj" and "'tb_frame'".
```

-----+	
traceback.tb_lineno	Gives the line number where the exception occurred
-----+	
traceback.tb_lasti	Indicates the precise instruction.
-----+	

The line number and last instruction in the traceback may differ from the line number of its frame object if the exception occurred in a "try" statement with no matching except clause or with a "finally" clause.

traceback.tb_next

The special writable attribute "tb_next" is the next level in the stack trace (towards the frame where the exception occurred), or "None" if there is no next level.

Changed in version 3.7: This attribute is now writable

Slice objects

Slice objects are used to represent slices for "__getitem__()" methods. They are also created by the built-in "slice()" function.

Special read-only attributes: "start" is the lower bound; "stop" is the upper bound; "step" is the step value; each is "None" if omitted. These attributes can have any type.

Slice objects support one method:

slice.indices(self, length)

This method takes a single integer argument *length* and computes information about the slice that the slice object would describe if applied to a sequence of *length* items. It returns a tuple of three integers; respectively these are the *start* and *stop* indices and the *step* or stride length of the slice. Missing or out-of-bounds indices are handled in a manner consistent with regular slices.

Static method objects

Static method objects provide a way of defeating the transformation of function objects to method objects described above. A static method object is a wrapper around any other object, usually a user-defined method object. When a static method object is retrieved from a class or a class instance, the object actually returned is the wrapped object, which is not subject to any further transformation. Static method objects are also callable. Static method objects are created by the built-in "staticmethod()" constructor.

Class method objects

A class method object, like a static method object, is a wrapper around another object that alters the way in which that object is retrieved from classes and class instances. The behaviour of class method objects upon such retrieval is described above, under `instance methods`. Class method objects are created by the built-in `"classmethod()"` constructor.

TYPESFUNCTIONS

=====

Functions

Function objects are created by function definitions. The only operation on a function object is to call it: `func(argument-list)`.

There are really two flavors of function objects: built-in functions and user-defined functions. Both support the same operation (to call the function), but the implementation is different, hence the different object types.

See [Function definitions](#) for more information.

TYPESMAPPING

=====

Mapping Types - "dict"

A **mapping** object maps **hashable** values to arbitrary objects. Mappings are mutable objects. There is currently only one standard mapping type, the **dictionary**. (For other containers see the built-in `"list"`, `"set"`, and `"tuple"` classes, and the `"collections"` module.)

A dictionary's keys are **almost** arbitrary values. Values that are not **hashable**, that is, values containing lists, dictionaries or other mutable types (that are compared by value rather than by object identity) may not be used as keys. Values that compare equal (such as `"1"`, `"1.0"`, and `"True"`) can be used interchangeably to index the same dictionary entry.

```
class dict(**kwargs)
class dict(mapping, /, **kwargs)
class dict(iterable, /, **kwargs)
```

Return a new dictionary initialized from an optional positional argument and a possibly empty set of keyword arguments.

Dictionaries can be created by several means:

* Use a comma-separated list of `"key: value"` pairs within braces:

```
"{'jack': 4098, 'sjoerd': 4127}" or '{"4098": 'jack', 4127:
'sjoerd'}"
```

* Use a dict comprehension: `"{}"`, `"{x: x ** 2 for x in range(10)}"`

* Use the type constructor: `"dict()"`, `"dict([('foo', 100), ('bar',`

```
200)})", "dict(foo=100, bar=200)"
```

If no positional argument is given, an empty dictionary is created. If a positional argument is given and it defines a "keys()" method, a dictionary is created by calling "__getitem__()" on the argument with each returned key from the method. Otherwise, the positional argument must be an *iterable* object. Each item in the iterable must itself be an iterable with exactly two elements. The first element of each item becomes a key in the new dictionary, and the second element the corresponding value. If a key occurs more than once, the last value for that key becomes the corresponding value in the new dictionary.

If keyword arguments are given, the keyword arguments and their values are added to the dictionary created from the positional argument. If a key being added is already present, the value from the keyword argument replaces the value from the positional argument.

Dictionaries compare equal if and only if they have the same "(key, value)" pairs (regardless of ordering). Order comparisons (<, <=, >=, >) raise "TypeError". To illustrate dictionary creation and equality, the following examples all return a dictionary equal to {"one": 1, "two": 2, "three": 3}:

```
>>> a = dict(one=1, two=2, three=3)
>>> b = {'one': 1, 'two': 2, 'three': 3}
>>> c = dict(zip(['one', 'two', 'three'], [1, 2, 3]))
>>> d = dict([('two', 2), ('one', 1), ('three', 3)])
>>> e = dict({'three': 3, 'one': 1, 'two': 2})
>>> f = dict({'one': 1, 'three': 3}, two=2)
>>> a == b == c == d == e == f
True
```

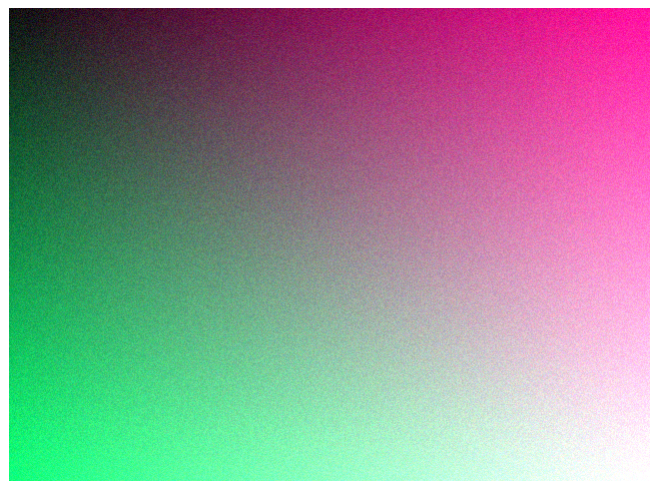
Providing keyword arguments as in the first example only works for keys that are valid Python identifiers. Otherwise, any valid keys can be used.

Dictionaries preserve insertion order. Note that updating a key does not affect the order. Keys added after deletion are inserted at the end.

```
>>> d = {"one": 1, "two": 2, "three": 3, "four": 4}
>>> d
{'one': 1, 'two': 2, 'three': 3, 'four': 4}
>>> list(d)
['one', 'two', 'three', 'four']
>>> list(d.values())
[1, 2, 3, 4]
>>> d["one"] = 42
>>> d
{'one': 42, 'two': 2, 'three': 3, 'four': 4}
>>> del d["two"]
>>> d["two"] = None
>>> d
{'one': 42, 'three': 3, 'four': 4, 'two': None}
```

Changed in version 3.7: Dictionary order is guaranteed to be insertion order. This behavior was an implementation detail of CPython from 3.6.

Dictionaries are generic over two types, signifying (respectively)



the types of the dictionary's keys and values.

These are the operations that dictionaries support (and therefore, custom mapping types should support too):

`list(d)`

Return a list of all the keys used in the dictionary **d**.

`len(d)`

Return the number of items in the dictionary **d**.

`d[key]`

Return the item of **d** with key **key**. Raises a "KeyError" if **key** is not in the map.

If a subclass of dict defines a method "`__missing__()`" and **key** is not present, the "`d[key]`" operation calls that method with the key **key** as argument. The "`d[key]`" operation then returns or raises whatever is returned or raised by the "`__missing__(key)`" call. No other operations or methods invoke "`__missing__()`". If "`__missing__()`" is not defined, "KeyError" is raised. "`__missing__()`" must be a method; it cannot be an instance variable:

```
>>> class Counter(dict):
...     def __missing__(self, key):
...         return 0
...
>>> c = Counter()
>>> c['red']
0
>>> c['red'] += 1
>>> c['red']
1
```

The example above shows part of the implementation of "collections.Counter". A different "`__missing__()`" method is used by "collections.defaultdict".

`d[key] = value`

Set "`d[key]`" to **value**.

`del d[key]`

Remove "`d[key]`" from **d**. Raises a "KeyError" if **key** is not in the map.

`key in d`

Return "True" if **d** has a key **key**, else "False".

`key not in d`

Equivalent to "not key in d".

`iter(d)`

Return an iterator over the keys of the dictionary. This is a

shortcut for "iter(d.keys())".

clear()

Remove all items from the dictionary.

copy()

Return a shallow copy of the dictionary.

classmethod fromkeys(iterable, value=None, /)

Create a new dictionary with keys from **iterable** and values set to **value**.

"fromkeys()" is a class method that returns a new dictionary. **value** defaults to "None". All of the values refer to just a single instance, so it generally doesn't make sense for **value** to be a mutable object such as an empty list. To get distinct values, use a dict comprehension instead.

get(key, default=None, /)

Return the value for **key** if **key** is in the dictionary, else **default**. If **default** is not given, it defaults to "None", so that this method never raises a "KeyError".

items()

Return a new view of the dictionary's items ("(key, value)" pairs). See the documentation of view objects.

keys()

Return a new view of the dictionary's keys. See the documentation of view objects.

pop(key, /)

pop(key, default, /)

If **key** is in the dictionary, remove it and return its value, else return **default**. If **default** is not given and **key** is not in the dictionary, a "KeyError" is raised.

popitem()

Remove and return a "(key, value)" pair from the dictionary. Pairs are returned in LIFO (last-in, first-out) order.

"popitem()" is useful to destructively iterate over a dictionary, as often used in set algorithms. If the dictionary is empty, calling "popitem()" raises a "KeyError".

Changed in version 3.7: LIFO order is now guaranteed. In prior versions, "popitem()" would return an arbitrary key/value pair.

reversed(d)

Return a reverse iterator over the keys of the dictionary. This is a shortcut for "reversed(d.keys())".

Added in version 3.8.

`setdefault(key, default=None, /)`

If **key** is in the dictionary, return its value. If not, insert **key** with a value of **default** and return **default**. **default** defaults to "None".

`update(**kwargs)`
`update(mapping, /, **kwargs)`
`update(iterable, /, **kwargs)`

Update the dictionary with the key/value pairs from **mapping** or **iterable** and **kwargs**, overwriting existing keys. Return "None".

"update()" accepts either another object with a "keys()" method (in which case "`__getitem__()`" is called with every key returned from the method) or an iterable of key/value pairs (as tuples or other iterables of length two). If keyword arguments are specified, the dictionary is then updated with those key/value pairs: "`d.update(red=1, blue=2)`".

`values()`

Return a new view of the dictionary's values. See the documentation of view objects.

An equality comparison between one "`dict.values()`" view and another will always return "False". This also applies when comparing "`dict.values()`" to itself:

```
>>> d = {'a': 1}
>>> d.values() == d.values()
False
```

`d | other`

Create a new dictionary with the merged keys and values of **d** and **other**, which must both be dictionaries. The values of **other** take priority when **d** and **other** share keys.

Added in version 3.9.

`d |= other`

Update the dictionary **d** with keys and values from **other**, which may be either a **mapping** or an **iterable** of key/value pairs. The values of **other** take priority when **d** and **other** share keys.

Added in version 3.9.

Dictionaries and dictionary views are reversible.

```
>>> d = {"one": 1, "two": 2, "three": 3, "four": 4}
>>> d
{'one': 1, 'two': 2, 'three': 3, 'four': 4}
>>> list(reversed(d))
['four', 'three', 'two', 'one']
>>> list(reversed(d.values()))
[4, 3, 2, 1]
>>> list(reversed(d.items()))
```

[('four', 4), ('three', 3), ('two', 2), ('o

